Ultimate Basic v1.5.6 — Language Manual

Complete language and CLI reference for Ultimate Basic, a BASIC-like language that compiles directly to 6502 machine code for the Commodore 64. Output: `.prg` files (VICE or real hardware), `.crt` cartridge images, and `.d64` disk images.

For a short project overview and build instructions see README.md.

> Building the `ub` compiler and the command-line options are covered in README.md. This manual documents the Ultimate Basic language itself.

Language reference

Variables and constants

All variables must be declared with `var` before use. Using an undeclared variable in an expression, assignment, `for` / `loop` counter, `inc` / `dec`, `input`, or `read` statement produces a compile-time error.

```
var x = 10              # 8-bit integer (default)
var ptr: word = $0400   # 16-bit — two zero-page bytes, usable as 16-bit address
var f: float = 3.5      # Q8.8 fixed-point — hi byte = integer, lo byte = fraction
var msg = "HELLO"       # string variable (pointer to inline PETSCII data)
var s: string = "TEXT"  # string with explicit type
var scores = array(10)  # byte array, 10 elements stored at $C000+
var moments  = array_word(8) # word array, 8 word elements stored at $C000+
const BORDER_ADDR = $D020 # compile-time constant (substituted inline, no ZP slot)
```

Keywords and identifiers are **case-insensitive**: `PRINT`, `Print`, and `print` are all valid.

Type	Width	Notes
`int`	8-bit	default for numeric literals
`word`	16-bit	two ZP bytes; can be used as address in `poke` / `peek`
`float`	16-bit Q8.8	hi byte = integer part (0–255), lo byte = fractional part
`string`	pointer	ZP pair → null-terminated PETSCII in code segment
`array(N)`	N bytes	byte elements; lives at `$C000+`, not in ZP
`array_word(N)`	N×2 bytes	word (16-bit) elements; lives at `$C000+`, not in ZP
`array(R, C, …)`	∏dims bytes	multi-dimensional (row-major); index `arr[r, c]`

Type	Width	Notes
`array_word(R, C, …)`	∏dims×2 bytes	multi-dimensional word array (row-major)

Reserved words

The following identifiers are **keywords** — they cannot be used as variable, constant, subroutine, function, parameter, or label names. All matching is case-insensitive (`END` , `end` , `End` all collide). Using a reserved word as a name usually produces a confusing error (a `var` line silently fails to declare, or an expression like `for i = 1 to times` folds to `Number(0)`) — so pick a different name.

Declaration & control flow
`var` , `const` , `sub` , `fn` , `type` , `endtype` , `return` , `call` ,
`label` , `goto` , `gosub` ,
`if` , `then` , `else` , `end` , `select` , `case` ,
`for` , `next` , `loop` , `times` , `to` , `step` , `while` , `repeat` , `until` ,
`break` , `continue` , `inc` , `dec` , `bye` , `exit` , `rem`

Types & type-related
`int` , `word` , `float` , `string` , `array` , `array_word`

Print & I/O
`print` , `spc` , `tab` , `at` , `input` , `chr$` , `str$` , `hex` , `bin` ,
`open` , `close` , `load` , `save` , `data` , `read` , `include` , `incbin`
(also `dec` — listed above as the decrement statement; the same token is used for the `dec(n, width)` print format)

Math & string built-ins
`abs` , `min` , `max` , `clamp` , `sgn` , `mod` , `rnd` , `sin` , `cos` ,
`and` , `or` , `xor` , `not` , `bnot` , `shl` , `shr` ,
`len` , `asc` , `val` , `str_to_int` , `numstr`

Memory & timing
`poke` , `peek` , `poke16` , `peek16` , `fill` , `memcopy` , `drawmem` ,
`wait` , `raster` , `delay` , `sys` , `asm`

Screen & text
`cls` , `fast` , `color` , `text` , `border` , `bg` , `screen` , `cursor` ,
`lowercase` , `uppercase` , `display` , `on` , `off` , `scroll` ,
`speed` , `badlines` , `turbo`

Bitmap & block graphics
`graphics` , `gcls` , `flip` , `plot` , `plot4` , `mplot` ,
`mline` , `mrect` , `mcircle` ,
`line` , `circle` , `circle4` , `rect` , `paint` , `erase` ,
`pen` , `multi` , `block`

Sprites
`sprite` , `sprdef` , `sprite_frame` , `sprite_x` , `sprite_y` , `sprhit` , `sprbghit` ,
`box_hit` , `chardef` , `charset` , `expand` , `priority`

Sound & music

`sid`, `sound`, `volume`, `music`, `play`, `pause`, `resume`, `stop`

Input devices

`getch`, `inkey`, `waitkey`, `joy`, `mouse_x`, `mouse_x_hi`, `mouse_y`, `mouse_btn`

Interrupts & vectors

`irq`, `irq_exit`, `nmi`, `nmi_exit`, `cia_timer`, `onerr`

Character maps & images

`map`, `map_tile`, `map_color`, `koala`, `show`, `hide`

REU (RAM expansion)

`reu`, `reudet`, `stash`, `fetch`

Style pitfalls — names that *look* free but are reserved

These common English words look like innocent identifiers but are already grabbed by the lexer. Rename to avoid silent breakage:

Reserved	Suggested rename
`end`	`stop_at`, `finish`, `last`
`stop`	`stop_at`, `halt`
`times`	`count`, `n`, `iters`
`screen`	`scraddr`, `screen_addr`
`border`, `bg`, `text`	`border_addr`, `bg_col`, `text_col`
`clamp`	`cap`, `bound`
`line`, `circle`, `rect`	`ln`, `circ`, `box_r`
`data`, `read`	`bytes`, `next_byte`
`load`, `save`, `open`, `close`	`load_file`, …
`map`, `show`, `hide`	`tilemap`, `reveal`, `conceal`
`play`, `pause`, `resume`, `stop`, `volume`	`play_song`, `vol`
`int`, `float`, `word`, `string`	`n_int`, `speed_f`, `addr`, `msg`

Symbolic tokens (`+ - * / = == != < > <= >= : , ; ( ) [ ] # $ % @`) are naturally not usable in identifiers.

Comments

```
# hash comment
rem this is also a comment
var x = 5  # inline comment
var x = 5 : var y = 6  # colon separates statements on one line
```

```
```

Operators

```
x = x + 1
y = a * b - c / 2
z = x and 15            # bitwise AND
w = a or b              # bitwise OR
v = a xor b             # bitwise XOR
m = x shl 3             # shift left
n = x shr 2             # shift right
r = x mod 40            # 8-bit modulo (remainder); SEC/SBC/BCS loop
b = bnot x              # bitwise NOT: x XOR 255 (complement all 8 bits)
```

Comparisons: `==` `!=` `<` `>` `<=` `>=` (return 1/0)

Increment / Decrement

```
inc x                   # x = x + 1  (INC zp — single instruction)
dec x                   # x = x - 1  (DEC zp — single instruction)
```

For `word` variables carry is handled: `inc` uses `INC lo; BNE skip; INC hi`; `dec` uses `LDA lo; BNE skip; DEC hi; DEC lo`.

Compound Assignments

```
x += 5                  # x = x + 5
x -= 3                  # x = x - 3
x *= 2                  # x = x * 2
x /= 4                  # x = x / 4
x and= 15               # x = x and 15   (bitwise AND)
x or= 64                # x = x or 64    (bitwise OR)
x xor= 255              # x = x xor 255  (bitwise XOR)
x shl= 2                # x = x shl 2
x shr= 1                # x = x shr 1
```

Print

```
print "HELLO"
print x
print x, y, "text"
print                     # blank line

print spc(5)             # print 5 space characters
print tab(20), "VALUE"  # move cursor to column 20, then print
print "A", spc(3), "B" # mix freely

print "A=" + a           # string + numeric var
print s1 + s2            # two string vars → sequential print
print "Hello " + "World" # two literals → compile-time fold
print chr$(13)           # print by PETSCII code
```

chr$

```
print chr$(65)           # output character with PETSCII code 65 ('A')
var c = chr$(n)          # store byte value n in variable c
print ">" + chr$(42)     # usable in string concat
```

Branching

```
if x == 1 then
  print "YES"
else
  print "NO"
end
```

Select / Case

```
select x
  case 1:
    print "ONE"
  case 2:
    print "TWO"
  else:
```

```
    print "OTHER"
end
```

`select expr` evaluates the expression once and compares it against each `case` value in order. The first matching case body is executed and control jumps to after `end`. The optional `else:` body runs if no case matches. All values must be 8-bit (0–255).

Loops

```
loop 5              # counted loop (5 iterations)
  print "HI"
end

times 5             # alias for loop N ... end
  print "HI"
end

loop                # infinite loop
  x = x + 1
  if x == 5 then continue end  # skip to next iteration
  if x == 100 then break end
end

var i = 0
for i = 1 to 10     # for..next (preferred)
  if i == 5 then continue end  # skip to increment step
  print i
next

for i = 0 to 20 step 2
  print i
next i              # variable name after 'next' is optional

for i = 10 to 1 step -1     # counting down — negative constant step is supported
  print i
next

for i = 20 to 0 step -2     # terminates correctly at i = 0 (11 iterations)
  print i
next i

var i = 0
loop i = 1 to 10     # legacy loop..end syntax — identical code
  print i
end

while x < 100
  x = x + 1
end
```

```
repeat              # do-while: body runs at least once
  x = x + 1
until x == 100      # exits when condition is true
```

Direction rules for `for` / `loop`:

Case	Behavior
`for i = 1 to 10` (default `step +1`, `from ≤ to`)	counts up, standard
`for i = 0 to 20 step 2` (positive step, `from ≤ to`)	counts up by 2
`for i = 10 to 1 step -1` (negative constant step)	counts down; body runs for i = 10, 9, ..., 1
`for i = 20 to 0 step -2` (down to zero)	terminates at i = 0 by detecting the ADC underflow (C=0) after the decrement; no infinite loop
`for i = 10 to 1` (no step, `from > to`)	**compile-time error**: `for-loop: from (10) > to (1) with default step +1 loops 0 times — use 'step -1' to count down`

The compiler picks the exit-branch encoding at compile time based on the sign of a constant `step`:

- Positive step (or default `+1`) → exit when `var > to` (unsigned `CMP` + `BCC` / `BEQ` fall-through to `JMP exit`).
- Negative constant step → exit when `var < to` (unsigned `CMP` + `BCS` to body), **plus** a post-increment `BCS loop_top ; JMP exit` that catches the wrap when `var` underflows below 0. That extra pair of instructions is what keeps `for i = N to 0 step -k` finite.

Non-constant `step` values (e.g. from a variable or expression) are treated as positive at compile time; if you need count-down with a runtime step value, split the loop or use a `while` construct.

Labels and goto

```
label main_loop
  x = x + 1
  if x < 10 then goto main_loop end
```

Forward `goto` (label defined later) is fully supported.

`gosub label` / `return` jump to a label and return back (JSR / RTS at machine code level).
The label must be a `label name` statement, not a `sub` — it has no parameters and shares the same zero-page scope. `gosub` supports forward references (label defined after `gosub`).

```
gosub draw_border
...
label draw_border
  # ... draw something ...
  return               # RTS — returns to the instruction after gosub
```

Subroutines

```
sub greet()
  print "HELLO!"
end

sub set_color(col)
  color border col
  color text   col
end

greet()              # call with parens
call greet           # call keyword (no parens)
set_color(6)
```

Parameters are passed via dedicated zero-page slots. No recursion (slots are static).
Typed parameters are supported: `sub draw(x, y:int)` or `sub copy(src:string)` — string parameters receive a 2-byte pointer so the callee can index the source string via `src[i]`.

Functions (return values)

```
fn square(x)
  return x * x
end

fn add(a, b)
  return a + b
end

fn clamp(val, lo, hi)
  if val < lo then return lo end
  if val > hi then return hi end
  return val
end
```

```
var s = square(9)         # s = 81  — fn call as expression
var c = clamp(joy, 0, 39) # c = clamped value
print add(10, 20)         # 30  — usable inline in print
```

Functions support an optional `: word` return type for 16-bit values:

```
fn get_addr(): word
  return $C000
end

var ptr: word = get_addr()  # ptr = $C000
poke ptr, 42                 # STA (ptr),Y — valid indirect addressing
var v = peek(ptr)            # LDA (ptr),Y
```

`fn` is emitted in pass 2 (same as `sub`), so function bodies are never executed at startup. Forward references are fully supported.

Arrays

```
var scores = array(8)    # 8 bytes at $C000

scores[0] = 100          # constant index → STA $C000
scores[i] = 99           # variable index → STA (ptr),Y
var v = scores[i]        # LDA (ptr),Y
print scores[2]          # usable inline in print

var moments = array_word(8)  # 16 bytes (8×2) at $C000+

moments[0] = $1234         # constant index → STA $C000 (lo), STA $C001 (hi)
moments[i] = $5678         # variable index → ASL A for stride; (ptr),Y × 2
var t: word = moments[1]   # LDA $C002, LDA $C003
```

Multi-dimensional arrays (new in 1.5.3)

Arrays may be declared with more than one dimension. They are stored **row-major** and indexed with a comma-separated subscript list. The total size is the product of all dimensions.

```
var grid = array(8, 8)     # 8×8 = 64 bytes at $C000
grid[1, 0] = 11            # row 1, col 0 → flat index 1*8+0 = 8 → STA $C008
```

```
grid[0, 1] = 22           # row 0, col 1 → flat index 1       → STA $C001
grid[r, c] = 33           # variable → computes r*8 + c, then STA (ptr),Y
var v = grid[r, c]        # LDA (ptr),Y

var wm = array_word(4, 4)  # 4×4 word array = 32 bytes
wm[1, 2] = $ABCD           # element 1*4+2 = 6 → bytes $C00C/$C00D

const ROWS = 3
const COLS = 5
var m = array(ROWS, COLS)  # dimensions may be compile-time constants
```

Row-major layout means the last subscript is contiguous: `grid[r, c]` sits at `base + r*COLS + c`. Any number of dimensions is supported (`array(a, b, c)`). When every subscript is a constant the address is folded at compile time to a single absolute store/load; a variable subscript emits the `row*stride + col` computation and an indexed `(ptr),Y` access. A single subscript into a multi-dimensional array is still allowed and treated as a flat/linear index (`grid[10]`). Dimensions must be compile-time constants (literals or `const` s), and the array must be declared before it is indexed.

Struct types (`type ... endtype` , new in 1.5.4)

Define a fixed layout of named fields with `type` , then allocate an array of instances the same way as a regular array. Field access uses `arr[idx].field` and works for both constant and variable indices.

```
type TEntity
  var x:  int      # 1 byte
  var y:  int      # 1 byte
  var hp: word     # 2 bytes  → element size = 4
endtype

const N = 8
var enemies: TEntity = array(N)   # allocates N × 4 = 32 bytes at $C000

enemies[0].x  = 5
enemies[0].y  = 10
enemies[0].hp = $0100
var v = enemies[0].hp             # read back — LDA $C002 / LDA $C003

var i: int = 2
enemies[i].x = 42                 # variable index → idx * elem_size + offset

# Float field (Q8.8 fixed-point)
type TBall
  var bx:  int     # 1 byte
  var by:  int     # 1 byte
  var vel: float   # 2 bytes Q8.8  → element size = 4
endtype
```

```
var balls: TBall = array(3)
balls[0].vel = 1.5             # stores Q8.8: hi=1, lo=128
var f: float = balls[0].vel    # read float field into float variable
print balls[0].vel             # prints "1.50"
```

Rules and constraints:

- Field types: `int` (1 byte), `word` (2 bytes LE), `float` (2 bytes Q8.8). `string` and nested `type` fields are not yet supported.
- Element size = sum of field widths (declaration order, no padding).
- Storage lives at `$C000+` alongside regular arrays, and shows up in the memory map as an ordinary array of the total byte size.
- Constant index → `LDA/STA absolute` at compile time.
- Variable index → shift-and-add multiply for `idx * elem_size`, then indexed `(ptr),Y` access. Element sizes that are powers of two (1, 2, 4, 8, 16) use bare `ASL A` chains; other sizes emit a short shift-add sequence via two scratch zero-page bytes.
- Sub / fn parameters of a struct type are not yet supported — pass the array and an index instead, e.g. `sub move(idx: int) ... enemies[idx].x = ...`.
- Default field values (`var fire: int = 2` inside the type) are parsed for compatibility with the proposal but not yet initialized at allocation time.
- Only one subscript is allowed on a struct-typed array; multi-dimensional struct arrays are not supported (use a computed flat index).

See `examples/type_demo.ub`.

16-bit (word) variables

```
var ptr: word = $0400    # two ZP bytes: lo=$00 hi=$04
poke ptr, 6              # STA (ptr),Y
var v = peek(ptr)        # LDA (ptr),Y
```

Bitmap graphics

```
graphics on              # VIC-II hires bitmap mode (320×200, 1bpp); bitmap at $2000
graphics on multi        # VIC-II multicolor bitmap mode (160×200, 2bpp, 4 colours/cell)
graphics off             # return to text mode

gcls                     # clear bitmap (fills $2000-$3FFF) + set video matrix colors

plot x, y                # set pixel at (x, y);  x: 0-319,  y: 0-199
plot erase x, y          # clear pixel (AND ~mask)
```

```
plot xor x, y            # toggle pixel (EOR mask) — flicker-free animation
circle x, y, r           # midpoint circle centered at (x, y) with radius r; clips off-screen points
line x1, y1, x2, y2      # Bresenham line from (x1,y1) to (x2,y2); x: 0-319, y: 0-199
line erase x1, y1, x2, y2  # clear pixels along the line (AND ~mask)
line xor x1, y1, x2, y2    # toggle pixels along the line (EOR mask)
rect x1, y1, x2, y2      # draw rectangle outline (4 edges); x: 0-319, y: 0-199
rect erase x1, y1, x2, y2  # clear rectangle outline (AND ~mask)
rect xor x1, y1, x2, y2    # XOR rectangle outline (EOR mask)
paint x, y               # 4-connected flood fill from (x, y); fills clear pixels bounded by set ones

color pen c              # set the hires drawing color (0-15); see below

# —— Multicolor bitmap (graphics on multi, 160×200) ——
mplot x, y, color        # set multicolor pixel (x: 0-159, y: 0-199, color: 0-3)
mline x1, y1, x2, y2, color   # multicolor line
mrect x1, y1, x2, y2, color   # multicolor rectangle outline
mcircle x, y, r, color        # multicolor circle
```

Both `graphics on` variants blank the display (`LDA $D011 / AND #$EF / STA $D011`) while switching VIC registers, then re-enable it in the target mode — prevents mode-switch glitches.

`x` may be the full `0–319` range. Coordinates above 255 are handled automatically (the helper adds the 9th X bit), so `plot`, `line`, `circle` and `rect` all reach the right edge of the screen. Use a `word` variable when an X coordinate can exceed 255.

Hires drawing color — `color pen`

In hires mode (320×200) color is **per 8×8 cell**, held in the video matrix (high nibble = foreground, low nibble = background), not per pixel. `color pen c` sets a persistent foreground color (0-15) that `plot`, `line`, `rect`, `circle` and `paint` stamp into the cell of every pixel they draw; the cell's background nibble is preserved. It stays in effect until the next `color pen`. The default is white (1), so programs that never call `color pen` look exactly as before.

```
graphics on
gcls
color pen 2              # red
line 0, 0, 100, 100
color pen 6              # blue
circle 160, 100, 40
display on
```

Multicolor shapes — `mline` / `mrect` / `mcircle`

In multicolor mode (`graphics on multi`, 160×200) each pixel picks one of four color sources via a 2-bit value (`%00` background `$D021`, `%01` screen hi nibble, `%10` screen lo nibble, `%11` color RAM). `mplot` sets a single such pixel; `mline`, `mrect` and `mcircle` draw shapes the same way — the trailing `color` argument is the 2-bit source (0-3), and the actual colors come from the cell palette (screen / color RAM) exactly as for `mplot`.

```
graphics on multi
gcls
mcircle 80, 100, 40, 1
mrect 10, 10, 150, 190, 2
mline 0, 0, 159, 199, 3
display on
```

`mline` / `mrect` / `mcircle` reuse the same Bresenham / midpoint routines as their hires counterparts, plotting each pixel through `mplot`; off-screen points (x ≥ 160 or y ≥ 200) are skipped.

Double-buffered bitmap (flicker-free)

```
graphics on double        # double-buffered hires bitmap (320×200)
gcls                      # clears the HIDDEN back buffer
line 0, 0, 319, 199       # all drawing (plot/line/circle/rect/paint/gcls) goes to the back buffer
flip                      # show the drawn buffer; redirect drawing to the other one
graphics off              # back to text mode (restores VIC bank 0)
```

`graphics on double` keeps **two** complete hires bitmaps and shows only finished frames, eliminating flicker without any XOR tricks — each frame you `gcls`, draw, then `flip`.

Buffer	Bitmap	Video matrix	VIC bank	`$DD00` low bits
A (front, shown first)	`$2000`	`$0400`	bank 0	`%11`
B (back, drawn first)	`$6000`	`$4400`	bank 1	`%10`

- All pixel commands write through a runtime **draw base** (a zero-page byte), so they automatically target whichever buffer is currently hidden.
- `flip` waits for the lower border (raster ≥ 251) before switching the VIC bank, so the swap is **tear-free**, then points the draw base at the now-hidden buffer.
- Typical loop: `gcls` → draw the frame → `flip`. Call `display on` once after the first `flip` so the first shown buffer is already complete.

Requirements / limits:

- Hires only (not `multi`). Sprites are not fetched from the back buffer's VIC bank.
- The back buffer uses `$4000-$7FFF` (matrix `$4400`, bitmap `$6000-$7FFF`), so the program's machine code must stay below `$4400`. Most demos are only a few KB, so this is

automatic; very large programs cannot use double buffering.
- On a stock 1 MHz C64 the per-frame full `gcls` (8 KB) limits the frame rate; on the Commodore 64 Ultimate raise `speed` for smooth high-rate animation.

See `examples/cube_demo.ub` — a tumbling 3D wireframe cube rendered flicker-free.

Block graphics (80×50)

```
graphics on block        # 80×50 block-pixel mode (text mode + custom 4-pixel charset @ $2800)
graphics off             # return to text mode
gcls                     # clear block playfield: screen RAM $0400-$07FF + color RAM $D800-$DBFF

plot4 x, y               # set block pixel at (x, y);  x: 0-79, y: 0-49
plot4 erase x, y         # clear block pixel at (x, y)
circle4 x, y, r          # draw midpoint circle in block pixels; clips to 80×50
```

A chunky low-res mode layered on standard 40×25 text. A 16-character custom charset copied to `$2800` encodes a 2×2 quadrant grid per character (bit3=TL, bit2=TR, bit1=BL, bit0=BR), so each text cell holds 2×2 block pixels → an effective 80×50 grid. No bitmap RAM is used (`$2000-$3FFF` stays free), making it faster than hires bitmap. `plot4` OR's the quadrant bit into the cell so overlapping pixels accumulate; `plot4 erase` clears it. `circle4` uses the same block-pixel helper to draw an outline circle in the 80×50 coordinate space. `gcls` clears both screen and color RAM. See `examples/block_demo.ub`.

Screen and color

```
cls                      # clear screen (KERNAL $E544)
cls fast                 # fast fill: screen RAM + color RAM + HOME

color text 14            # text color register $0286
color border 6           # $D020
color bg 0               # $D021
color pen 2              # hires drawing color (0-15) for plot/line/rect/circle/paint

screen 0, 0, 65          # write char 65 ('A') to screen RAM at col 0, row 0 ($0400)
screen 10, 5, ch         # col 10, row 5 — col/row can be variables
screen 5, 3, 42, 7       # char 42 at col 5, row 3, color 7 (writes color RAM $D800 too)
screen x, y, ch, col     # all four arguments as variables

display on               # re-enable VIC display ($D011 DEN bit)
display off              # blank display

lowercase                # CHR$(14) → switch VIC-II to lowercase/uppercase charset
uppercase                # CHR$(142) → switch VIC-II back to uppercase/graphics charset

cursor 20, 10            # move cursor to col 20, row 10 (KERNAL PLOT $FFF0)
```

```
cursor x, y              # column from variable x (0-39), row from y (0-24)

print at 20, 10, "HELLO" # cursor(20,10) + print in one statement (no trailing newline)
print at x, y, "Score:", score  # any mix of exprs
print at 0, 0            # position only (no text, no newline)

scroll x 3               # set horizontal fine scroll: $D016 bits 0-2 = 3 (0-7)
scroll y 2               # set vertical fine scroll:   $D011 bits 0-2 = 2 (0-7)
scroll x n               # value can be a variable or expression (masked to bits 0-2)
scroll x 7 narrow        # set fine scroll and force 38-column mode (hide edge column)
scroll x 0 wide          # set fine scroll and restore 40-column mode
scroll row 12 left       # shift screen RAM row 12 left by one character
```

`lowercase` emits `LDA #$0E; JSR $FFD2` at runtime. String literals compiled after `lowercase` have their case swapped automatically — uppercase source chars are stored in the PETSCII lowercase slot (`$61+`) and lowercase source chars in the uppercase slot (`$41+`) — so `"Hello World"` source displays as **Hello World** on screen. `uppercase` emits `LDA #$8E; JSR $FFD2` and reverts to direct mapping. `cls` does **not** reset the charset mode.

`scroll x n` writes `(n AND 7)` into bits 0-2 of `$D016` (preserving bits 3-7).
`scroll x n narrow` writes the fine-scroll bits and clears `$D016` bit 3 (38-column mode).
`scroll x n wide` writes the fine-scroll bits and sets `$D016` bit 3 (40-column mode).
`scroll y n` writes `(n AND 7)` into bits 0-2 of `$D011` (preserving bits 3-7).
`scroll row R left` shifts one constant screen row left; write the new rightmost character with `screen 39, R, ch`.
Useful for smooth hardware scrolling: decrement each frame from 7 down to 0, shift screen RAM, reset to 7.

`screen col, row, char [, color]` writes directly to screen RAM (`$0400 + row*40 + col`) and optionally to color RAM (`$D800 + row*40 + col`). Constant col/row: address computed at compile time.

Ultimate 64 — CPU Speed

```
speed 4             # set CPU to 4 MHz  (reads $D031, updates bits 0-3, writes back)
speed 48            # 48 MHz  (maximum speed on U64)
speed max           # same as speed 48  (alias)
speed off           # back to 1 MHz  (alias for speed 1)

badlines on         # enable badline timing  ($D031 bit 7 = 0, default C64 behaviour)
badlines off        # disable badline timing ($D031 bit 7 = 1, more CPU cycles)

var t = turbo()     # 1 if turbo is active (bits 0-3 of $D031 != 0), 0 if at 1 MHz
```

Available MHz values: `1, 2, 3, 4, 5, 6, 8, 10, 12, 14, 16, 20, 24, 32, 40, 48`.
Constant values are rounded down to the nearest available speed at compile time.
Variable values are treated as a raw speed index (0–15) and OR'd into bits 0-3 of `$D031`.

$D031 index	MHz (U64)	MHz (U64 Elite-II)
0	1	1
3	4	4
6	8	10
11	20	24
15	48	64

Requires **U64 Turbo Control** set to `U64 Turbo Registers` or `Turbo Enable Bit` in the U64 config menu. On a regular C64 or emulator without the register, the `poke` to `$D031` is silently ignored.

Keyboard

```
var key = getch()         # busy-wait on $FFE4 until key; returns PETSCII code
var k   = inkey()         # non-blocking: returns PETSCII code, or 0 if no key pressed
waitkey                   # wait until any key is pressed (CIA1 matrix scan; works during IRQ)
var k   = waitkey()       # same but returns raw $DC01 column bits (0 bit = key pressed)
var j = joy(2)            # read joystick port 2; returns inverted bits 0-4
var j = joy(1)            # read joystick port 1
                          # bit0=up(1) bit1=down(2) bit2=left(4) bit3=right(8) bit4=fire(16)
var mx = mouse_x()        # 1351 mouse: accumulated X position (helper: charge+delay+delta)
var my = mouse_y()        # 1351 mouse: accumulated Y position (EOR #$FF inverted)
var mb = mouse_btn()      # mouse buttons: reads $DC01 — bit0=left (fire), bit1=right (up pin)
```

`mouse_x()` / `mouse_y()` internally run a helper subroutine that handles: CIA1 `$DC00` capacitor charging, ~516-cycle delay, SID POT X/Y register read (`$D419` / `$D41A`), signed 7-bit delta computation, Y-axis `EOR #$FF` inversion, and accumulated position tracking (permanent ZP state, 8 bytes). The helper samples POT twice per frame. `mouse_y()` does **not** call the helper (only `mouse_x()` does) — it just reads the cached `accum_y` to avoid double-update.

The helper maintains a 9-bit X accumulator (`accum_x` + `accum_x_hi` with carry/borrow). Use `var sx: word = mouse_x()` — the compiler stores both lo and hi bytes, so the `sprite` command correctly handles `$D010` for X positions beyond 255.

The helper stores its state in ZP (`$02` – `$09`). The init flag must be zero before the first call — zero the area with `fill $02, 7, 0` early in your program. Also disable CIA1 interrupts (`poke $DC0D, $7F`) to prevent the KERNAL IRQ from touching `$DC00` during keyboard scanning — that disturbs the POT reading.

See `examples/mouse_demo.ub` for a working 1351 mouse demo with sprite tracking and button feedback.

Exit

```
bye                    # JSR $E544 (clear screen), clear STOP flag, RTS to BASIC
exit                   # alias for bye
```

Timing

```
wait 50                # wait 50 raster-line transitions (~3.2 ms)
wait raster 100        # spin until $D012 == 100 (raster-split effects)
delay 1                # wait 1 PAL frame (1/50 s ≈ 20 ms)
delay 20               # wait 20 frames ≈ 0.4 s; n can be a variable (0–255)
```

`delay N` counts N complete PAL frames using raster line 200 as the frame boundary.

SID Sound

```
sound 0, $1CAD, 25     # voice 0, freq $1CAD (≈ middle C PAL), 25 frames duration
sound 1, freq_word, 50 # voice 1, freq from word var, 50 frames (1 s at 50 Hz)
sound 2, 0, 0          # voice 2, silence

sid volume 15          # master volume full ($D418 = $0F); range 0-15
sid volume 0           # silence (master volume = 0)
sid stop               # zero all 25 SID registers ($D400–$D418) — complete silence
```

`sound <channel>, <freq>, <duration>` — duration in PAL frames (1/50 s each).
Fixed ADSR: attack/decay `$09`, sustain/release `$F0`, sawtooth waveform.
Master volume `$D418` always set to `$0F`.

`sid volume N` writes N to `$D418`. Bits 0-3 = volume (0-15), bits 4-7 = filter mode.
`sid stop` emits a 10-byte zero-fill loop — faster than 25 individual pokes.

Music playback

`music play/stop/pause/resume` is a high-level alternative to the manual `sys sid_init` / `cia_timer` setup.
Requires a prior `load sid` statement (defines `sid_init` / `sid_play`).

```
load sid "tune.sid"       # embed SID file (defines sid_init / sid_play)

music play                # initialise sub-tune 0 + start CIA1 50 Hz IRQ
```

```
    music play 1             # start from sub-tune 1 (song number 0-based)
    music stop               # stop playback + zero all 25 SID registers ($D400-$D418)
    music pause              # disable CIA1 timer A IRQ (music freezes, SID unchanged)
    music resume             # re-enable CIA1 timer A IRQ (continues from pause point)
```

Statement	Effect
`music play [n]`	call `sid_init(n)`, configure CIA1 timer A at 19 656 cycles (~50 Hz PAL), install IRQ wrapper
`music stop`	disable CIA1 IRQ + zero all 25 SID registers
`music pause`	disable CIA1 IRQ (SID output remains frozen)
`music resume`	re-enable CIA1 IRQ (continues from pause point)

The IRQ wrapper (emitted once at the end of the program) does: ACK CIA1 timer A → `JSR sid_play` → `JMP $EA81`.

Sprites

```
    sprite 0, x, y, $2000    # sprite 0: set X, Y position and data pointer
    sprite 0, x, y           # without data pointer (keeps existing)
    sprite on  0             # enable sprite 0 ($D015 |= bit0)
    sprite off 0             # disable sprite 0 ($D015 &= ~bit0)
    sprite color 0, 7        # sprite 0 color = yellow ($D027)
    sprite multicolor 0, on  # enable multicolor mode for sprite 0 ($D01C |= bit0)
    sprite multicolor 0, off # disable multicolor mode ($D01C &= ~bit0)
    sprite expand x 0, on    # double width ($D01D |= bit0)
    sprite expand x 0, off   # normal width ($D01D &= ~bit0)
    sprite expand y 0, on    # double height ($D017 |= bit0)
    sprite expand y 0, off   # normal height ($D017 &= ~bit0)
    sprite priority 0, on    # behind background ($D01B |= bit0)
    sprite priority 0, off   # in front of background ($D01B &= ~bit0)
    var h = sprite_hit()     # sprite-sprite collision ($D01E, cleared on read)
    var b = sprite_bg_hit()  # sprite-background collision ($D01F, cleared on read)
    var x = sprite_x(0)      # read sprite 0 X position (lo byte, $D000)
    var y = sprite_y(0)      # read sprite 0 Y position ($D001)
    sprite_frame 0, $2000          # select one static sprite image
    sprite_frame 0, $2000, frame   # select an animation frame from a frame sequence
```

X supports full 9-bit range (0–319): use a `word` variable for runtime values > 255.
Sprite data pointer: `data_addr` must be 64-byte aligned; stored as `addr >> 6` at `$07F8+id`.

Sprite animation with `sprite_frame`

`sprite_frame` is the command used to change the displayed image of a sprite during animation. It changes only the sprite's data pointer; it does **not** move the sprite, enable it, or advance frames automatically.

Store the animation images consecutively, with each 63-byte sprite image occupying one 64-byte-aligned slot. Pass the zero-based animation frame as the third argument:

```
sprite_frame sprite_id, first_frame_address, frame_number
```

For example, with a base address of `$2000`, frame 0 uses `$2000`, frame 1 uses `$2040`, frame 2 uses `$2080`, and so on. The program controls animation timing and wrapping:

```
var frame = 0
loop
  sprite_frame 0, $2000, frame
  frame = frame + 1
  if frame == 4 then frame = 0 end
  delay 5
end
```

The two-argument form, `sprite_frame id, address`, simply selects one static sprite image and remains backward compatible. Sprite position is still controlled by `sprite id,x,y`.

Software bounding-box collision

```
var touching = box_hit(left1, top1, right1, bottom1,
                       left2, top2, right2, bottom2)
```

`box_hit()` performs an axis-aligned bounding-box (AABB) test and returns `1` when the two rectangles overlap or touch, otherwise `0`. Unlike `sprite_hit()` and `sprite_bg_hit()`, it does not read or clear VIC-II collision registers. The eight arguments are arbitrary 8-bit expressions, so boxes may be smaller than the visible sprite artwork or may describe non-sprite game objects. Coordinates are inclusive; keep `left <= right` and `top <= bottom`.

Sprite definition

```
sprdef 0
  $00,$3C,$00,  $00,$FF,$00,  $03,$FF,$C0,  $07,$FF,$E0,
  $0F,$FF,$F0,  $0F,$FF,$F0,  $1F,$FF,$F8,  $1F,$FF,$F8,
  $1F,$FF,$F8,  $0F,$FF,$F0,  $0F,$FF,$F0,  $07,$FF,$E0,
```

```
    $03,$FF,$C0,  $00,$FF,$00,  $00,$3C,$00,  $00,$00,$00,
    $00,$00,$00,  $00,$00,$00,  $00,$00,$00,  $00,$00,$00,
    $00,$00,$00
  end
```

`sprdef id ... end` embeds 63 sprite bytes at the next 64-byte-aligned address in the code segment, emits a `JMP` over them, and automatically sets `$07F8+id = data_addr >> 6`. To use the same shape for multiple sprites, read back the pointer:

```
var pg = peek($07F8)   # pointer set by sprdef 0
poke $07F9, pg         # copy to sprites 1-7
```

Character tile maps (`.ubmap`)

```
map load "levels/world.ubmap"
map draw map_x, map_y       # draw a 40x25 viewport to screen/color RAM

var tile = map_tile(x, y)   # read character code from the map
map set x, y, 42            # change character code in writable map data

var shade = map_color(x, y) # read cell color (0 when map has no color data)
map color x, y, 7           # change cell color when color data is present
```

`map load` resolves the filename relative to the `.ub` source file, validates it at compile time, and embeds its character and optional color arrays in writable program RAM. A later `map load` replaces the active map for subsequent map commands.

`map load` also accepts a VisualAssembler `me-map` `.bin` export directly:

```
map load "map-multicolor.bin"
```

The first 1000 bytes become the 40×25 character map and the next 1000 bytes become cell colors. Multicolor mode and `$D021-$D023` are read from the VisualAssembler metadata trailer, so no `.ubmap` conversion is required.

`map draw map_x, map_y` copies a 40×25 viewport beginning at the specified map cell to screen RAM `$0400` and, when present, color RAM `$D800`. The map must contain the whole requested viewport: keep `map_x <= width-40` and `map_y <= height-25`. Coordinates and dimensions are currently 8-bit (0–255).

Normal maps clear the text multicolor bit in `$D016`. Multicolor maps set it and load their three global colors into `$D021`, `$D022`, and `$D023`. Character maps contain character codes, not charset pixels; combine them with `charset` / `chardef` or another charset-loading method. In multicolor text mode, cell colors 8–15 select multicolor characters according to the VIC-II rules.

UBMP version 1 binary format

All multibyte integers are little-endian:

Offset	Size	Meaning
0	4	ASCII magic `UBMP`
4	1	Version, currently `1`
5	1	Flags: bit 0 = color array present, bit 1 = multicolor text mode
6	2	Map width, 1–255 cells
8	2	Map height, 1–255 cells
10	1	Background 0 (`$D021`), low nibble used
11	1	Multicolor 1 (`$D022`), low nibble used
12	1	Multicolor 2 (`$D023`), low nibble used
13	width×height	Row-major character codes
following	width×height	Optional row-major color nibbles when flag bit 0 is set

An UBMP file must have the exact length implied by its header. Invalid magic, version, dimensions, or length produces a compile-time error.

Koala Painter image import

```
koala load "pictures/title.kla"  # validate and embed at compile time
koala show                       # enter bitmap multicolor mode
koala hide                       # return to the default text display
```

`koala load` accepts a standard 10003-byte Koala file (`$6000` load address plus 10001 data bytes) or a raw 10001-byte payload. Paths are relative to the `.ub` file. The payload contains 8000 bitmap bytes, 1000 screen bytes, 1000 color nibbles, and one background-color byte.

The compiler stores it at `$6000-$8710`. `koala show` copies the bitmap to `$2000`, the screen matrix to `$0400`, colors to `$D800`, and enables bitmap multicolor mode. `koala hide` clears bitmap/multicolor mode and restores the default text layout.

Generated code and helpers must finish below `$2000`, because the displayed bitmap overwrites `$2000-$3F3F`; the compiler reports an error otherwise. Koala import cannot currently be combined with `load sid` in the same program.

Custom charset

```
charset $3800           # set base address for chardef (default $3800)

chardef 65              # redefine character 65 ('A')
  $18,$3C,$66,$7E,$66,$66,$66,$00
end

chardef 66              # fewer than 8 bytes are zero-padded
  $7C,$66,$7C,$66,$7C
end
```

`charset base` sets the destination address used by all subsequent `chardef` statements. `chardef id ... end` embeds 8 bytes inline in the code segment (preceded by a `JMP` to skip over them), then copies them to `charset_base + id*8` at runtime. Values must be compile-time constants; use `%` for binary literals (`%00011000`).

To activate a custom charset in VIC-II, set the character generator address via `$D018`:

```
charset $3800
chardef 1  $FF,$81,$81,$81,$81,$81,$81,$FF  end  # box border
poke $D018, $1A     # screen at $0400, charset at $3800 (bank 0)
```

Memory

```
poke $D020, 2           # STA $D020
poke addr_var, 6        # STA (addr_var),Y  — if addr_var is word type
var v = peek($D012)     # LDA $D012
var v = peek(addr_var)  # LDA (addr_var),Y  — if addr_var is word type

var w: word = peek16($C000)   # read 16-bit little-endian: lo=$C000, hi=$C001
poke16 $0314, $EA81           # write 16-bit little-endian: lo→$0314, hi→$0315
poke16 ptr, w                 # word var as address; word var as value
```

`peek16(addr)` reads two consecutive bytes (lo, hi) as a `word`. `poke16` writes lo then hi.

Disk I/O

```
load "PROGRAM"           # KERNAL LOAD: loads file from device 8 to its native address
load "DATA", $C000       # loads file to a specific address
load "DATA", ptr         # addr from word variable

save "DATA", $C000, 4096 # KERNAL SAVE from $C000, 4096 bytes → device 8
save "PROG", start, len  # addr and len from word/int variables
```

`load` calls KERNAL `SETNAM` + `SETLFS` + `LOAD` (`$FFBD` / `$FFBA` / `$FFD5`).
Without address: secondary address 0 (file's own 2-byte header used as load address).
With address: secondary address 1 (file loaded to specified location).
`save` calls `SETNAM` + `SETLFS` + `SAVE` (`$FFBD` / `$FFBA` / `$FFD8`). Requires both `addr` and `len` .

SID Music

```
load sid "tune.sid"           # embed SID music at its native load address
load sid "tune.sid", $2000    # override: embed at $2000 regardless of SID header
```

`load sid` reads a PSID or RSID file at **compile time**, strips the header, and appends the raw music bytes to the output `.prg` . After `load sid` , two compile-time constants become available:

Constant	Description
`sid_init`	Init routine address — call once with A = song number (0-based)
`sid_play`	Play routine address — call every frame (50 Hz PAL) from an IRQ handler

Both constants work anywhere a constant address is accepted: `sys` , `irq` , `poke` , expressions.

Typical usage:

```
load sid "music.sid"

sub music_irq()
  poke $D019, $FF         # ACK VIC raster IRQ
  sys sid_play            # advance one frame of music
  irq_exit                # JMP $EA81: restore A/X/Y + RTI (proper IRQ exit)
end

sys sid_init, 0         # initialise SID chip: A=0 → first sub-tune
irq music_irq, $C0      # raster IRQ at line $C0 → 50 Hz on PAL
```

```
sid volume 15          # master volume on
```

Notes:

- SID data is placed **after** all generated code, padded with zeros up to the load address. The compiler warns if the SID load address would overlap generated code.
- PSID v1 and v2 are supported. If the SID header's load address is 0, the first two data bytes are used as the address (PRG-style, little-endian).
- Only one `load sid` per program is meaningful (the last one wins).

Serial channel file I/O

```
open 1, 8, 2, "MYFILE"  # open logical file 1, device 8, secondary 2, name "MYFILE"
open 2, 4, 7             # open printer (device 4), no filename
open ch, dev, sec        # channel, device, secondary from variables

print# 1, "HELLO"        # send "HELLO"+CR to logical file 1
print# ch, x, "text"     # any mix of vars, strings — same as print but to file

close 1                  # close logical file 1
close ch                 # channel from variable
```

`open` calls `SETNAM` ($FFBD) + `SETLFS` ($FFBA) + `OPEN` ($FFC0). Without a filename, SETNAM is called with length 0.
`print#` routes output via `CHKOUT` ($FFC9), CHROUT per char (+ trailing CR), then `CLRCHN` ($FFCC).
`close` puts the channel number in A and calls `CLOSE` ($FFC3).

Input

```
input score              # read up to 3 digits from keyboard → 8-bit int var
input "Name: ", name     # optional prompt string, then read line → string var
input "Score: ", score   # prompt + int input
```

`input` uses KERNAL BASIN (`$FFCF`) for blocking, echoed line input with DEL support.

- **Int var**: accepts only `0` – `9` , max 3 chars; converts to 8-bit value on CR.
- **String var**: accepts up to 30 chars; stores as null-terminated string; ZP pair updated.

Float / Fixed-Point

`float` variables use Q8.8 fixed-point format: the high byte is the integer part (0–255) and the low byte is the fractional part (0/256 … 255/256).

```
var f: float = 3.5       # 3.5 → hi=3, lo=128 (= 0x0380)
var g: float = 0         # integer 0 is promoted to 0.0 automatically

f = 1.5                  # Q8.8 literal assignment
f = f + 1.5              # 16-bit Q8.8 arithmetic (result: 3.0)
f = f + g                # float + float

var n = int(f)           # extract integer part (hi byte) → 8-bit int
print f                  # prints as "N.DD" (e.g. 3.5 → "3.50", 1.25 → "1.25")
```

Operation	Example	Notes
Literal	`3.5`, `0.25`, `1.0`	parsed as Q8.8 at compile time
Integer promotion	`f = 5`	stores 5.0 (hi=5, lo=0)
Add/sub	`f + 1.5`, `f - g`	16-bit Q8.8 arithmetic
Extract int	`int(f)`	returns hi byte as 8-bit int
Print	`print f`	format "N.DD", always 2 fractional digits

Caveat: Arithmetic overflow wraps at 255.255 (no saturation). Multiplication and division of two float variables are not yet supported — use `int()` + integer arithmetic for those cases.

Math functions

```
var a = abs(x - 20)      # two's-complement absolute value
var b = min(x, 39)       # 8-bit minimum
var c = max(x, 0)        # 8-bit maximum
var s = sgn(score)       # 0 = zero, 1 = positive (1–127), $FF = negative (128–255)
var r = rnd()            # LCG pseudo-random 0-255; seed from raster line
var r = rnd(10)          # LCG pseudo-random 0-9 (rnd() mod n; result 0..n-1)
var s = sin(angle)       # sine: angle 0-255 (full circle), returns 0-255 (center=128)
var c = cos(angle)       # cosine = sin(angle+64)
var v = clamp(x, 0, 39)  # 8-bit unsigned clamp: result = max(lo, min(hi, val))

print hex(n)             # print as 2-digit uppercase hex
print bin(n)             # print as 8-bit binary string
```

String functions

```
var n = len(msg)          # length of null-terminated string var (0–255)
var c = asc(msg)          # PETSCII code of first character (0 if empty)
var c = asc("A")          # compile-time: constant PETSCII code
var n = val(s)            # runtime: parse decimal PETSCII string → 8-bit int (e.g. "042" → 42)
var c = msg[i]            # string character at index i: PETSCII code of msg[i]
msg[i] = c                # write PETSCII byte c to string at index i — STA (ptr),Y
msg[0] = 72               # constant index → LDY #0; STA (ptr),Y
```

Number formatting

```
print hex(n)              # print as 2-digit uppercase hex
print bin(n)              # print as 8-bit binary string
print dec(n, 4)           # right-justified decimal in a field of 4 chars (e.g. 42 → "  42")
print dec(n, width)       # width can also be a variable
```

`dec(n, width)` pads the number on the left with spaces to fill `width` characters.
If the number has more digits than `width`, it is printed without padding (no truncation).
In non-print contexts `dec(n, w)` evaluates to `n` unchanged (same as `hex` / `bin`).

REU (RAM Expansion Unit)

```
var ok = reu_present()   # 1 if REU detected, 0 if not (write/read test on $DF04)
var ok = reudet()        # alias for reu_present()

reu stash c64addr, bank, reu_addr, len  # copy C64 → REU
reu fetch c64addr, bank, reu_addr, len  # copy REU → C64
reu swap  c64addr, bank, reu_addr, len  # swap between C64 and REU
```

`reu_present()` performs a write/read-back test on REU register `$DF04`. Without an REU the write is lost (open bus), so the read-back differs — reliably detects presence without touching any side-effecting command register.

Parameter	Width	Notes
`c64addr`	16-bit	C64 RAM start — constant, `word` var, or 8-bit expr
`bank`	8-bit	REU bank number (0–7 for a 512 KB unit)
`reu_addr`	16-bit	Offset within the REU bank

Parameter	Width	Notes
`len`	16-bit	Bytes to transfer (`0` = 65 536 in REU hardware)

REU registers: `$DF01` command (`$B0` stash / `$B1` fetch / `$B2` swap),
`$DF02–$DF03` C64 addr, `$DF04–$DF05` REU offset, `$DF06` bank, `$DF07–$DF08` length.
Transfer is synchronous (CPU halted during DMA).
Requires a real REU or VICE: **Settings** → **Hardware** → **RAM Expansion Module**.

Memory utilities

```
fill screen 32          # fill screen RAM $0400-$07FF with value 32 (space char)
fill color 1            # fill color RAM $D800-$DBFF with value 1 (white)

fill $0400, 1000, 32    # fill 1000 bytes starting at $0400 with value 32
fill addr, 256, 0       # addr can be word var
fill ptr, len_word, val  # all operands can be expressions / word vars

memcopy $C000, $0400, 256   # copy 256 bytes from $C000 → $0400
memcopy src_ptr, dst_ptr, 40 # word vars for source and destination

drawmem $C000, $0400, 8, 10, 40 # blit 8×10 rect from $C000 → screen at $0400, stride 40
drawmem src_ptr, dst_ptr, w, h, 40 # word vars for src/dst
```

Both `fill` and `memcopy` support 16-bit lengths (0–65535). Use `word` variables for lengths > 255.

`drawmem src, dst, width, height, stride` copies a 2-D rectangular block. `src` is read linearly (packed rows); `dst` advances by `stride` bytes between rows — use `40` ($28) for the C64 screen or color RAM (40 columns). Width, height and stride are all 8-bit values.
`src` and `dst` may be constants, `word` variables, or 8-bit expressions.

Raster IRQ

```
irq my_handler          # raster IRQ at line 0, handler = sub name or address
irq my_handler, 100     # raster IRQ at raster line 100
irq $C800, 200          # handler at fixed address
irq addr_word           # handler address from a word variable
```

Sets up a raster IRQ via the BASIC soft vector (`$0314` / `$0315`): disables CIA1 timer IRQ, ACKs pending VIC IRQ, writes raster line to `$D012` , enables VIC raster IRQ (`$D01A=$01`), writes handler address, and re-enables interrupts.

The handler **must** end with `sys $EA81` (KERNAL end-of-IRQ) — plain `RTS` or `RTI` will corrupt the stack. ACK the VIC IRQ first:

```
sub my_handler()
  poke $D019, $FF      # ACK VIC IRQ
  # ... work here ...
  sys $EA81            # JMP to KERNAL end-of-IRQ
end
```

Forward references are supported (`irq my_handler` before the sub is defined).

NMI handler

```
nmi my_nmi               # set NMI vector $0318/$0319 to handler sub or address

sub my_nmi()
  # ... NMI work here ...
  nmi_exit               # JMP $FE47 — proper NMI exit (restores A/X/Y + RTI)
end
```

`nmi handler` writes the handler address to the NMI soft vector (`$0318` / `$0319`). The hardware NMI vector at `$FFFA` points to the KERNAL NMI routine which branches through `$0318` . The handler **must** end with `nmi_exit` (emits `JMP $FE47`) — using plain `RTI` will corrupt the stack. Forward references supported.

CIA1 timer IRQ

```
cia_timer 19656, my_handler   # CIA1 timer A: fires every 19656 cycles (~50 Hz PAL)
cia_timer period, handler      # period can be a variable or expression
```

Sets up CIA1 timer A as a periodic IRQ source via the BASIC soft vector (`$0314` / `$0315`):

1. SEI — disable interrupts
2. `$DC0D = $7F` — disable all CIA1 IRQs
3. Load 16-bit period lo→ `$DC04` , hi→ `$DC05`
4. Write handler address to `$0314` / `$0315`
5. `$DC0D = $81` — enable CIA1 timer A IRQ
6. `$DC0E = $01` — start timer A in continuous mode
7. CLI — re-enable interrupts

The handler must end with `irq_exit` (or `sys $EA81`) and should ACK the CIA1 IRQ:

```
sub my_handler()
  poke $DC0D, $01      # ACK CIA1 timer A IRQ (read also clears it)
  # ... work here ...
  irq_exit             # JMP $EA81: restore A/X/Y + RTI
end
```

PAL timing: clock = 985 248 Hz. Period for 50 Hz = 985 248 / 50 = 19 705 cycles ≈ `$4CC9`. Forward references supported.

Error handling

```
onerr goto err_handler   # set KERNAL I/O error vector ($0300/$0301) to a label

...

label err_handler
  print "I/O ERROR"
  bye
```

`onerr goto label` writes the label address (lo, hi) to KERNAL locations `$0300` and `$0301`. When a KERNAL I/O error occurs (e.g. a failed `load` or `open`) the KERNAL executes `JMP ($0300)`, which branches to the label. Forward references (label defined after `onerr goto`) are supported.

Compile-time file embedding

```
incbin "sprites.bin"            # embed raw binary bytes at current code position
incbin "charset.bin", $2000     # embed at an absolute address, padding as needed
include "defs.ub"        # inline another .ub source file (lexed+parsed in place)
```

Data / Read

```
data 1, 2, 3, 255        # constant byte table
read varname             # load next byte into varname (auto-declares if needed)
```

All `data` values are collected at compile time. A 2-byte ZP pointer is automatically allocated and initialised at program start. Each `read` advances the pointer.

Inline assembly

```
sys $FFD2                # JSR $FFD2
sys $FFD2, 7             # LDA #7 ; JSR $FFD2  (pass byte value in A register)
irq_exit                 # JMP $EA81 — proper IRQ handler exit (restore A/X/Y + RTI)
asm $EA, $EA             # inline raw bytes (NOP NOP) — legacy form
asm {
  ; Full 6502 mnemonics and addressing modes
  LDA #$07               ; immediate
  STA $0286              ; absolute
  LDA $50                ; zero-page  ($50 ≤ $FF → ZP auto-selected)
  STA $0400,X            ; absolute,X indexed
  LDA ($50),Y            ; (indirect),Y
  LDA ($50,X)            ; (indirect,X)
  JSR $FFD2              ; subroutine call
  JMP $C000              ; absolute jump
  JMP ($FFFC)            ; indirect jump

  CLC
  ADC #1
  SEC
  SBC #1

  TAX                    ; implied / transfer
  ASL A                  ; accumulator (also just: ASL)
  LSR A
  ROL
  ROR

  ; Branches — operand is an absolute address; offset is computed automatically
  BNE loop               ; forward or backward branch to local label
  BEQ done

loop:
  NOP
done:
  RTS

  ; #<label / #>label — lo / hi byte of a label address
  LDA #<handler
  STA $0314
  LDA #>handler
  STA $0315

  ; * — current assembly location
  JMP *

  ; Raw hex bytes (backward-compatible with old asm { $xx ... } syntax)
```

```
    $EA $EA                ; two NOP bytes
  }
```

Addressing modes:

Syntax	Mode	Bytes	Example
(no operand)	Implied	1	`NOP`, `RTS`
`A`	Accumulator	1	`ASL A`, `LSR`
`#value`	Immediate	2	`LDA #$07`
`$zz` (0–255)	Zero-page	2	`LDA $50`
`$zz,X`	ZP,X	2	`LDA $50,X`
`$zz,Y`	ZP,Y	2	`LDX $50,Y`
`$xxxx`	Absolute	3	`LDA $0400`
`$xxxx,X`	Absolute,X	3	`LDA $0400,X`
`$xxxx,Y`	Absolute,Y	3	`LDA $0400,Y`
`($xxxx)`	Indirect	3	`JMP ($FFFC)`
`($zz,X)`	(Indirect,X)	2	`LDA ($50,X)`
`($zz),Y`	(Indirect),Y	2	`LDA ($50),Y`
`label`	Relative	2	`BNE label` (branches only)

- `$zz` (1–2 hex digits, value ≤ 255) selects zero-page if the instruction supports it; otherwise auto-upgrades to absolute. Use `$00xx` (4 digits) to force absolute.
- Branch operands are absolute addresses; the relative byte offset is computed automatically.
- Local labels (`name:`) are scoped to the `asm { }` block. Forward branches resolved in pass 2.
- `#<label` / `#>label` yield the lo / hi byte of a label's address.
- `*` yields the current instruction address, so `JMP *` assembles as a self-loop.
- Lines starting with `$`, `%`, or a digit are emitted as raw bytes (backward-compatible).
- Inside `asm { }`, comments are `;` or `//` to end of line. (`#` is the immediate prefix, not a comment.)

Mixing `asm { }` with subroutine parameters

Parameter names are **not accessible** inside `asm { }` blocks. Use UltimateBasic statements to move values into known locations before the `asm { }` block:

```
sub set_colors(border_col, bg_col)
  poke $D020, border_col   # UltimateBasic resolves the ZP address
  poke $D021, bg_col
  asm {
    ; values are already in $D020 / $D021
    LDA $D020
```

```
    }
end
```

For routines whose entire body is assembly — especially IRQ handlers that must cross-reference each other — put all handlers in a **single top-level `asm { }` block** in the main program. Labels in the same block share scope, so `irq1` and `irq2` can reference each other freely. See `examples/raster_irq_demo.ub`.

String ↔ integer

```
numstr score, $0340      # writes "042\0" at $0340 (always 3 digits, zero-padded)
var n = str_to_int("42") # compile-time: Expr::Number(42)

print str$(score)               # print 8-bit int as 3-digit decimal string ("000"-"255")
print "Score: " + str$(score)   # usable in string concat print context
var s: string = str$(n)         # assign str$() result to a string var (shared static buffer)
```

`str$(n)` converts an 8-bit value to a 3-character decimal string (always 3 digits with leading zeros, e.g. `5` → `"005"`, `42` → `"042"`, `255` → `"255"`) followed by a null terminator. The result pointer is stored in a permanent ZP pair allocated at compile time.

> **Note:** `str$(n)` uses a single shared 4-byte static buffer. Calling `str$(n)` again overwrites the previous result. For concurrent display of multiple values, use `numstr` to write to separate absolute addresses.

Examples

File	Description
`examples/features.ub`	const, label/goto, poke/peek, rnd, math functions
`examples/new_features.ub`	sub params, arrays, word vars, string vars
`examples/bitmap_demo.ub`	320×200 bitmap, plot, graphics on/off
`examples/block_demo.ub`	80×50 block graphics, plot4, circle4, graphics on block
`examples/joystick_demo.ub`	joystick reading, sprite movement
`examples/mux_demo.ub`	raster sprite multiplexer (3 windows × 8 sprites = 24)
`examples/orbit_demo.ub`	24-sprite orbit with pulsating radius and random colors
`examples/plasma_demo.ub`	plasma-effect bitmap with raster bar border animation
`examples/sprite_data.ub`	sprdef shape data (included by other demos)

File	Description
`examples/sprite_mux_orbit.ub`	24-sprite orbit demo with sprdef + precomputed positions
`examples/sprite_orbit_demo.ub`	8 hardware sprites in circular orbit via sin/cos table
`examples/reu_bitmap_demo.ub`	REU stash/fetch with bitmap graphics
`examples/sid_music_demo.ub`	SID music player with raster IRQ and keyboard exit
`examples/tenprint.ub`	5 TENPRINT maze implementations with menu; demos `lowercase` charset mode
`examples/countdown_demo.ub`	Count-down `for..next` with negative step, incl. `for i = 20 to 0 step -2`
`examples/countdown_errors_demo.ub`	Compile-time error for default-step `from > to`
`examples/explicit_demo.ub`	`--explicit` CLI-flag demo with fully typed `var`, `sub`, `fn`
`examples/explicit_errors_demo.ub`	Loose code that builds without `--explicit`, errors with it

CLI reference

```
ub build <input.ub> [OPTIONS]

  -o, --output <file>   Output .prg or .crt file (default: <input>.prg)
  -v, --verbose         Show zero-page layout and code hex dump
  --no-stub             Skip the BASIC SYS stub (code loads at $0801)
  --debug               Also produce .sym, .dbg and .vs debugger files
  --asm                 Also produce a readable 6502 codegen .asm listing
  --d64 [file]          Also produce a .d64 disk image;
                          without a filename defaults to <output>.d64
  --add <file>          Add an extra file to the .d64 disk image;
                          may be repeated for multiple files
  --explicit            Require :type on every var / sub-param / fn-param
  -h, --help            Show help
```

CRT export

Ultimate Basic can also write a Magic Desk type-19 cartridge image when the output filename ends in `.crt`:

```
ub build demo.ub -o demo.crt
```

The compiler wraps the generated PRG in a banked CRT container using the same layout as VisualAssembler's CRT export: a 64-byte cartridge header, an 8×8K

Magic Desk image, and a boot loader in bank 0 that copies the PRG payload into C64 RAM before jumping to the program entry point. The `.prg` and `.d64` paths continue to work as before.

Explicit-type mode

Pass `--explicit` to force every declaration site to carry a `:type` annotation:

```
ub build game.ub --explicit
```

Without the flag, Ultimate Basic accepts both `var x = 5` (type inferred from the initializer or defaulted to `int`) and `var x: int = 5`. With the flag, only the annotated form compiles — the loose form becomes a compile-time error.

What it enforces:

Declaration form	Without `--explicit`	With `--explicit`
`var name = expr` (no `:type`)	ok — type inferred	error
`var name: int = expr` (or word/float/string)	ok	ok
`var arr = array(N)`	ok — implicitly `array`	ok — unchanged
`var arr = array_word(N)`	ok — implicitly `array_word`	ok — unchanged
`const NAME = value`	ok	ok — unchanged
`sub foo(a, b)` (untyped params)	ok	error per untyped param
`sub foo(a: int, b: int)`	ok	ok
`fn foo(a): int` (untyped param)	ok	error on the param
`fn foo(a: int): int`	ok	ok

Constants and array declarations are always accepted — their type is fixed by the declaration form, so `:type` would be redundant.

Example error output:

```
$ ub build myprog.ub --explicit
Compilation errors:
  line 14: 'explicit' mode: 'var loose' has no type — use 'var loose: int|word|float|string'
  line 16: 'explicit' mode: parameter 'a' has no type — use 'a: int|word|float|string'
```

The flag is a build-time switch — nothing in the source needs to change to opt in or out. Add it to your Makefile / build script to enforce typed style project-wide, or drop it for exploratory scripts. See `examples/explicit_demo.ub` and `examples/explicit_errors_demo.ub`.

Debug files

Use `--debug` to generate debugger symbols together with the program:

```
ub build demo.ub --debug
```

The compiler writes the files next to the `.prg`, using the output file's stem:

File	Format and purpose
`demo.sym`	KickAssembler-compatible symbol source, suitable for importing into assembler source
`demo.dbg`	C64Debugger/RetroDebugger KickAssembler debug-dump containing the program segment and labels
`demo.vs`	VICE monitor command file containing `al` commands for address labels

All three exports include `program_start`, `program_end`, variables, arrays, subroutines, and BASIC labels known after code generation. For example, load the VICE symbols with its `-moncommands demo.vs` command-line option or the monitor's `ll "demo.vs"` command.

The current `.dbg` export provides segment and address-symbol information. It does not yet include instruction-to-source-line mappings for source-level stepping.

Assembly codegen listing (new in 1.5.2)

Use `--asm` to write a readable 6502 source listing next to the PRG:

```
ub build demo.ub --asm
```

For an output named `demo.prg`, this creates `demo.asm`. The listing is produced from metadata collected while Ultimate Basic generates the machine code; it is not merely a disassembly of the completed PRG. It contains:

- `; UB:` comments marking the generated byte range of each emitting UB statement;
- named constants for zero-page variables and `$C000+` arrays, including their types/sizes;
- final names and addresses for subroutines and BASIC labels;
- generated `loc_xxxx` labels for relative branches and in-program `JMP` / `JSR` targets;
- normal 6502 mnemonics and operands, with compiler symbols substituted where known;
- the exact C64 address and emitted bytes beside every instruction;
- compiler-generated helpers and embedded code in their final memory order;
- `.byte` output for bytes that do not decode as supported 6502 instructions.

Compiler helpers receive descriptive names such as `ub_helper_plot`, `ub_helper_line_erase`, `ub_helper_print_hex`, and `ub_helper_music_irq`. Known embedded data—including `data` statements, map character/color arrays, sprite and character definitions, the sine table, load filenames, `incbin` files, SID music, and Koala payloads—is emitted as named `.byte` regions. Long zero-filled address gaps use KickAssembler's `.fill` directive.

Example excerpt:

```
.label x                 = $02 ; int

* = $080D

ub_start:
    cld                         ; $080D: D8
    ; UB: var x
    lda  #$01                   ; $080E: A9 01
    sta  x                      ; $0810: 85 02
```

The address/byte comments make the listing easy to compare with the `.prg`. The output uses KickAssembler syntax (`.label`, `.byte`, `.fill`, and `* = origin`) so it can be assembled again; machine-code comments do not affect the result.

`--add` requires `--d64`. The compiled `.ub` program is always the first file on the disk; each `--add` file is appended after it. File names on the disk are derived from the source file stem, uppercased (e.g. `music.prg` → `MUSIC`).

After a successful build the compiler always prints a memory map:

```
demo.ub → demo.prg  (386 bytes)

  Load:    $080D – $0989

  Variables (zero page):
    score    ZP:$08   byte
    lives    ZP:$0A   byte
    msg      ZP:$0C   string
    ptr      ZP:$0E   word

  Subroutines:
    greet    $0900
    show     $0912

  Arrays ($C000+):
    sprites  $C000   8 bytes
```

With `-v` the output additionally shows the internal ZP allocations and a full hex dump.

Known limitations

Feature	Limitation
Integer arithmetic	8-bit unsigned (0–255); `word` vars hold 16-bit values
Subroutines	No recursion — ZP parameter slots are statically allocated
String vars	Read-only after init; assignment replaces the pointer, not the data
String concat runtime	`s1 + s2` prints sequentially — no heap allocation or length tracking
`rnd()` / `rnd(n)`	Simple LCG, not cryptographic; period = 256
`abs()` / `sgn()` / `min()` / `max()`	8-bit values only; `abs` / `sgn` treat values as signed (bit 7 = negative → `abs` two's-complements, `sgn` returns `$FF`); `min` / `max` are unsigned (0–255)
`plot`	Out-of-range pixels are silently clipped (Y ≥ 200 or X ≥ 320 → no-op)
`mplot`	No bounds checking — x must be 0–159, y must be 0–199
`mline` / `mrect`	Multicolor; x: 0–159, y: 0–199. Off-screen pixels wrap (no clipping) — keep coordinates in range
`mcircle`	Multicolor; clips off-screen points (x ≥ 160 or y ≥ 200 are skipped)
`color pen`	Hires only; sets the foreground nibble of touched cells (background preserved). Has no effect in block mode (`plot4` / `circle4`)
`rect`	No bounds checking — x: 0–319, y: 0–199; x1≤x2 and y1≤y2 not enforced (degenerate/inverted rects produce undefined output)
`plot4`	No bounds checking — x must be 0–79, y must be 0–49 (block mode)
`circle4`	Clips off-screen block pixels; useful radius is roughly 0–49 in 80×50 block mode
`chr$`	No PETSCII↔ASCII mapping — n is passed as-is to CHROUT
`music play`	Requires `load sid`; only one CIA1 wrapper is emitted (last `music play` wins)
`graphics on double`	Hires only; uses `$4000-$7FFF` for the back buffer, so program code must stay below `$4400`; not combinable with sprites or multicolor
Error reporting	Compile-time only; `onerr goto` handles KERNAL I/O errors at runtime